

NuMicro® Family
Arm® -based Microprocessor

Nuvoton OpenWrt 24.10 Project

User Manual

The information described in this document is the exclusive intellectual property of Nuvoton Technology Corporation and shall not be reproduced without permission from Nuvoton.

Nuvoton is providing this document only for reference purposes of NuMicro microprocessor based system design. Nuvoton assumes no responsibility for errors or omissions.

All data and specifications are subject to change without notice.

For additional information or questions, please contact: Nuvoton Technology Corporation.

www.nuvoton.com

Table of Contents

1 OVERVIEW	3
1.1 Feature List.....	3
2 DEVELOPMENT ENVIRONMENT SETUP	4
2.1 Docker Environment.....	4
2.1.1 Installing Docker (Example for Ubuntu 20.04)	4
2.1.2 Setting Up the Nuvoton Build Environment	4
2.2 Native Linux Environment	5
3 BUILD IMAGE.....	7
3.1 Update Feeds Scripts	7
3.2 OpenWrt Configurations.....	7
3.3 Linux Kernel Configurations.....	11
3.4 Build Image.....	12
3.4.1 MA35D1/MA35D0/MA35D05K/MA35H0 Platform.....	12
3.4.2 Error Fix	12
3.5 Deploy Image	12
3.5.1 NuWriter GUI Tool	12
3.5.2 NuWriter Command	15
3.6 Secure Boot.....	16
3.6.1 Configuration.....	16
3.6.2 Build Image	16
3.6.3 Deploy OTP Key File	17
3.7 emWin (TODO).....	18
3.7.1 Copy the emWin Source to OpenWrt	18
3.7.2 Configuration.....	19
3.7.3 Samples	19
4 TEST OPENWRT	20
4.1 Booting Messages.....	20
4.2 Network Settings	20
4.3 LuCI Web Interface	21
4.4 Firmware Update	23
5 REVISION HISTORY	24

1 OVERVIEW

The OpenWrt Project is a Linux operating system targeting embedded devices. Instead of creating a single, static firmware, OpenWrt provides a fully writable filesystem with package management. This Nuvoton OpenWrt implementation is based on OpenWrt 24.10.2. For fundamental concepts and detailed documentation, please refer to the official OpenWrt website at <https://openwrt.org/>.

1.1 Feature List

The Nuvoton OpenWrt 24.10 release includes the following key components and features:

- Linux Kernel
- LuCI (Web Interface)
- U-boot
- Arm-Trusted-Firmware (TF-A)
- Optee-OS
- Python3-Nuwriter (Programming Tool)

2 DEVELOPMENT ENVIRONMENT SETUP

To develop projects within the OpenWrt environment, the following requirements must be met:

- Host System: A Linux distribution (e.g., recent releases of Ubuntu, Debian, Fedora, or CentOS).
- Disk Space: A minimum of 20 GB of free disk space.
- Build Packages: Appropriate dependencies and tools installed on the host system.

Nuvoton provides two methods for building images: Native Linux and Docker.

While building directly on a Linux host is supported, system updates may introduce dependency conflicts that lead to build errors. Therefore, Nuvoton recommends the Docker-based environment. Unlike a native setup, Docker provides a containerized, isolated environment specifically for building images. This ensures consistency and prevents build tools from affecting the host operating system.

2.1 Docker Environment

Docker is an open-source project based on Linux containers. Similar to virtual machines, containers provide an isolated environment but are more portable, resource-efficient, and share the host operating system's kernel. Using Docker is the recommended way to quickly set up the OpenWrt build environment.

2.1.1 Installing Docker (Example for Ubuntu 20.04)

1. Update the package list:

```
$ sudo apt-get update
```

2. Install prerequisite packages:

```
$ sudo apt install apt-transport-https ca-certificates curl software-properties-common
```

3. Add Docker's official GPG key:

```
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

4. Add the Docker repository to APT sources:

```
$ sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu focal stable"
```

5. Install Docker Engine:

```
$ sudo apt-get update
$ sudo apt-get install docker-ce docker-ce-cli containerd.io
```

2.1.2 Setting Up the Nuvoton Build Environment

Nuvoton provides a set of scripts to automate the Docker image creation and management. You can find the source files at: [MA35D1 Docker Script](#).

1. **Build the Docker Image:** Run the build script to generate the Docker image and set up the shared folder. The host folder will be mounted at /home/\$USER/shared inside the container.

```
$ ./build.sh
```

2. **Enter the Docker Container:** Use the join.sh script to access the container environment. Upon success, the command prompt will change to [user name]@[container id]:~\$.

```
$ ./join.sh
ma35d1_test
```

```
test@575f27a6d251:~$
```

3. **Create a Working Directory:** Inside the container, create a dedicated folder within the shared directory to store the OpenWrt source code. This ensures your data persists on the host machine.

```
test@575f27a6d251:~$ mkdir shared/openwrt
```

```
test@575f27a6d251:~$ cd shared/openwrt
```

4. **Configure Git Identity:** Before using Git for the first time, you must configure your global identity. These settings are required for Git to function properly during the build and download process.

```
test@575f27a6d251:~$ git config --global user.email "test@test.test.test"
```

```
test@575f27a6d251:~$ git config --global user.name "test"
```

5. **Download the OpenWrt Source Code:** Clone the Nuvoton OpenWrt project repository from GitHub to your working directory.

```
test@575f27a6d251:~$ git clone https://github.com/OpenNuvoton/Nuvoton-OpenWrt-24.10.git
```

For further information on Docker, please refer to the [Official Docker Documentation](#).

2.2 Native Linux Environment

If you choose a native Linux environment for building images, several prerequisite packages must be installed before starting the OpenWrt project. For a comprehensive list of dependencies for other Linux distributions, please refer to the [OpenWrt Build System Documentation](#).

1. **Install OpenWrt Dependencies (Ubuntu 22.04):** Run the following command to install the required packages for the OpenWrt project:

```
$ sudo apt install build-essential gawk gcc-multilib flex git gettext libncurses5-dev libssl-dev python3-distutils rsync unzip zlib1g-dev
```

2. **Install OpenWrt Dependencies (Ubuntu 20.04 and older):** For older Ubuntu releases, use the following command:

```
$ sudo apt install build-essential ccache ecj fastjar file g++ gawk \
gettext git java-propose-classpath libelf-dev libncurses5-dev \
libncursesw5-dev libssl-dev python python2.7-dev python3 unzip wget \
python-distutils-extra python3-setuptools python3-dev rsync subversion \
swig time xsltproc zlib1g-dev
```

3. **Install Component-Specific Dependencies:** In addition to the core OpenWrt packages, sub-projects require specific tools:

- For Arm Trusted Firmware (TF-A):

```
$ sudo apt install libssl-dev device-tree-compiler
```

- For Optee-os:

```
$ pip3 install pycryptodomex pyelftools
```

- For Python3-Nuwriter:

```
$ pip3 install pyusb usb crypto ecdsa crcmod tqdm pycryptodome
```

4. **Download the OpenWrt Source Code:** Once the environment setup is complete, clone the Nuvoton OpenWrt project repository:

```
$ git clone https://github.com/OpenNuvoton/Nuvoton-OpenWrt-24.10.git
```

3 BUILD IMAGE

This section provides the detailed information along with the process for building an image.

3.1 Update Feeds Scripts

After download OpenWrt source is completed, execute following commands to update the OpenWrt feeds script.

```
$ ./scripts/feeds update -a
$ ./scripts/feeds install -a
```

If encounter following error, it means that your machine does not have the required certificate path to the CA of OpenWrt.

```
fatal: unable to access 'https://git.openwrt.org/feed/routing.git/':
gnutls_handshake() failed: The TLS connection was non-properly terminated.
```

To avoid this issue, you can run following command to tell git to ignore the cert procedure.

```
$ export GIT_SSL_NO_VERIFY=1
```

3.2 OpenWrt Configurations

For the MA35 series, this BSP supports MA35D1, MA35D0, MA35D05K and MA35H0 four targets. The MA35D1 platform supports IoT board, IoTv1 board, SOM board and HMI board, the MA35D0 platform supports IoT board, the MA35D05K platform supports IoTv1 board, and the MA35H0 platform supports HMI board. Please use the correct setting for the target board. For example, to use the MA35D1 IoT board, user needs to follow the steps below:

In folder Nuvoton-OpenWrt-24.10, use the file Nuvoton/config/config_ma35d1_iot as the OpenWrt configuration file.

```
$ cp Nuvoton/config/config_ma35d1_iot .config
```

Run “make menuconfig” to configure OpenWrt. Confirm the Target System is “Nuvoton MA35D1”, the Subtarget is the “MA35D1 IoT”, and the Target Profile with the correct memory size and booting storage, as shown in Figure 3-1.

```
$ make menuconfig
```

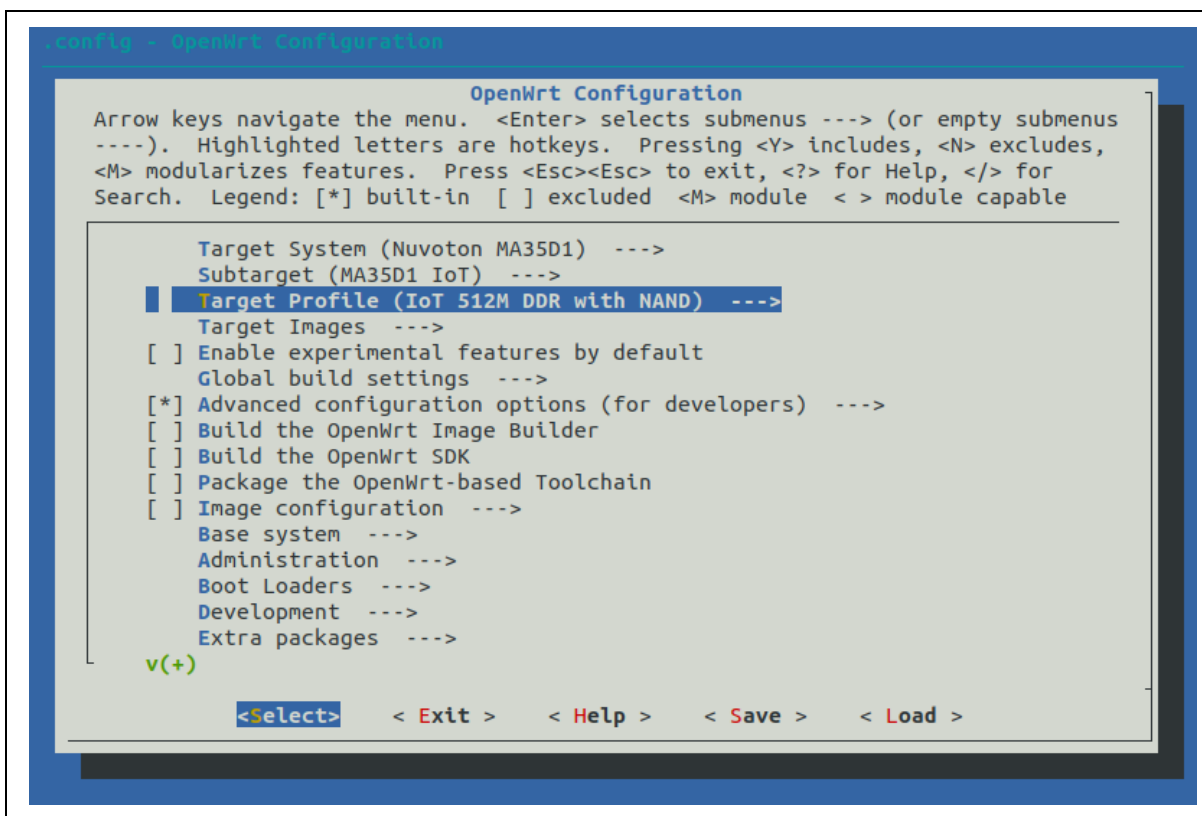


Figure 3-1 Select Target, Subtarget and Profile for MA35D1

In the Boot Loaders page, the Optee-os, TF-A, and U-Boot options should be enabled, as shown in Figure 3-2. Essential hardware initialization parameters—such as CPU speed, PMIC configuration, and DDR settings—are defined specifically within the TF-A options to ensure a proper secure boot process. Please notice if the “Set Custom DDR header file automatically” option is enabled, you should confirm the “MA35 Chip version” setting is correct.

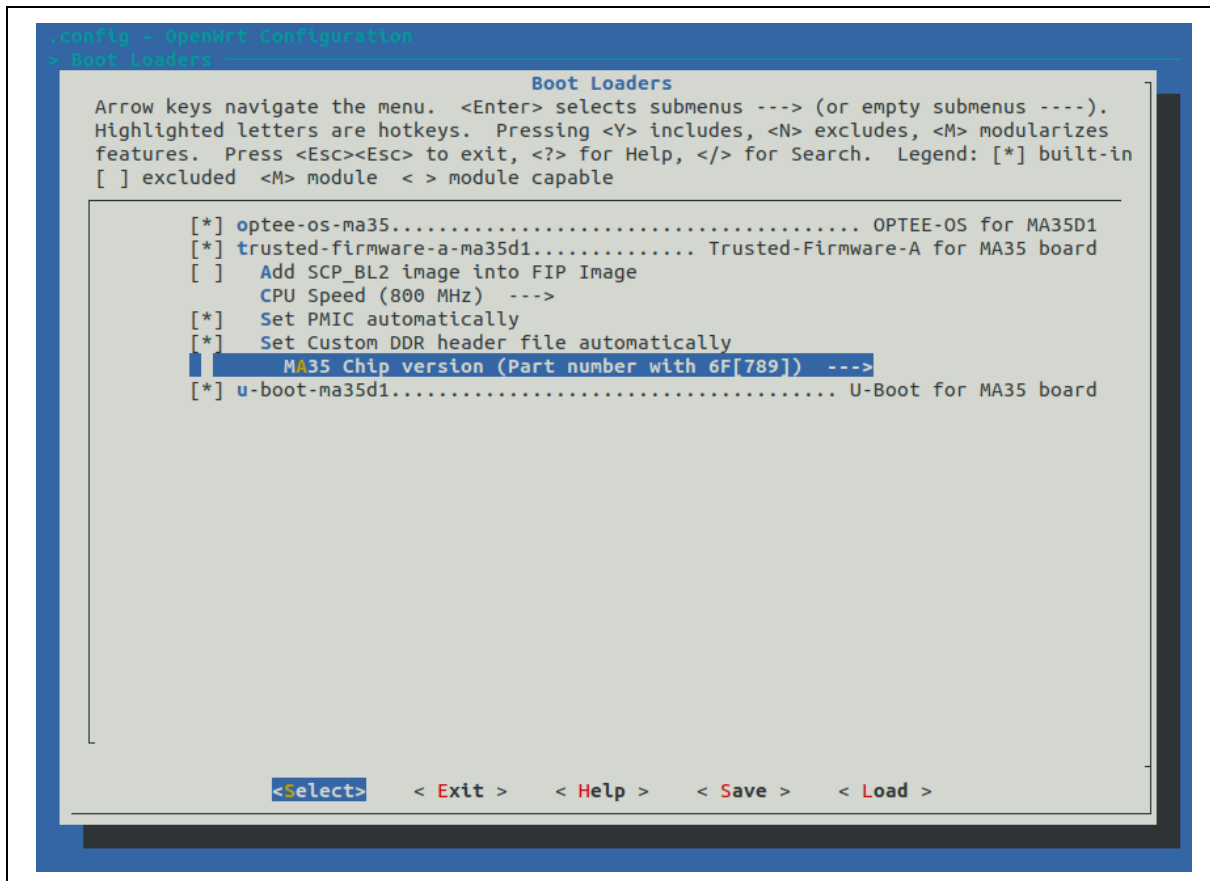


Figure 3-2 Boot Loaders Configurations for MA35D1

When booting from an SDCARD or eMMC, developers can choose between a squashfs + ext4 OverlayFS or an ext4-only root filesystem. This selection is controlled in menuconfig under the Target Images menu via the "SD card rootfs_data (overlay) partition size" option, as shown in Figure 3-3. Setting this value to a non-zero number instructs the build system to generate the squashfs + ext4 OverlayFS, while setting it to zero will build the ext4-only filesystem.

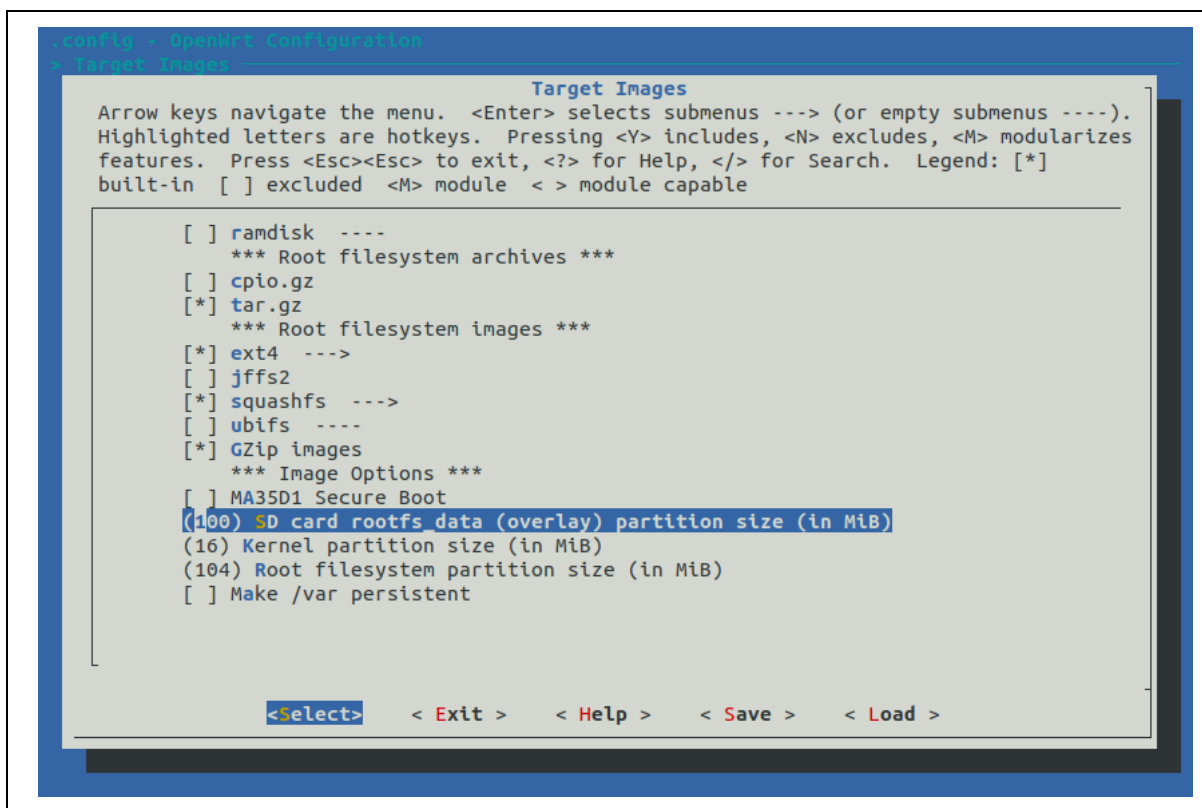


Figure 3-3 Configuring the Root Filesystem for SDCARD and eMMC Boot

In Advanced configuration options page, specify the git repository and a branch/commit to clone Linux kernel source.

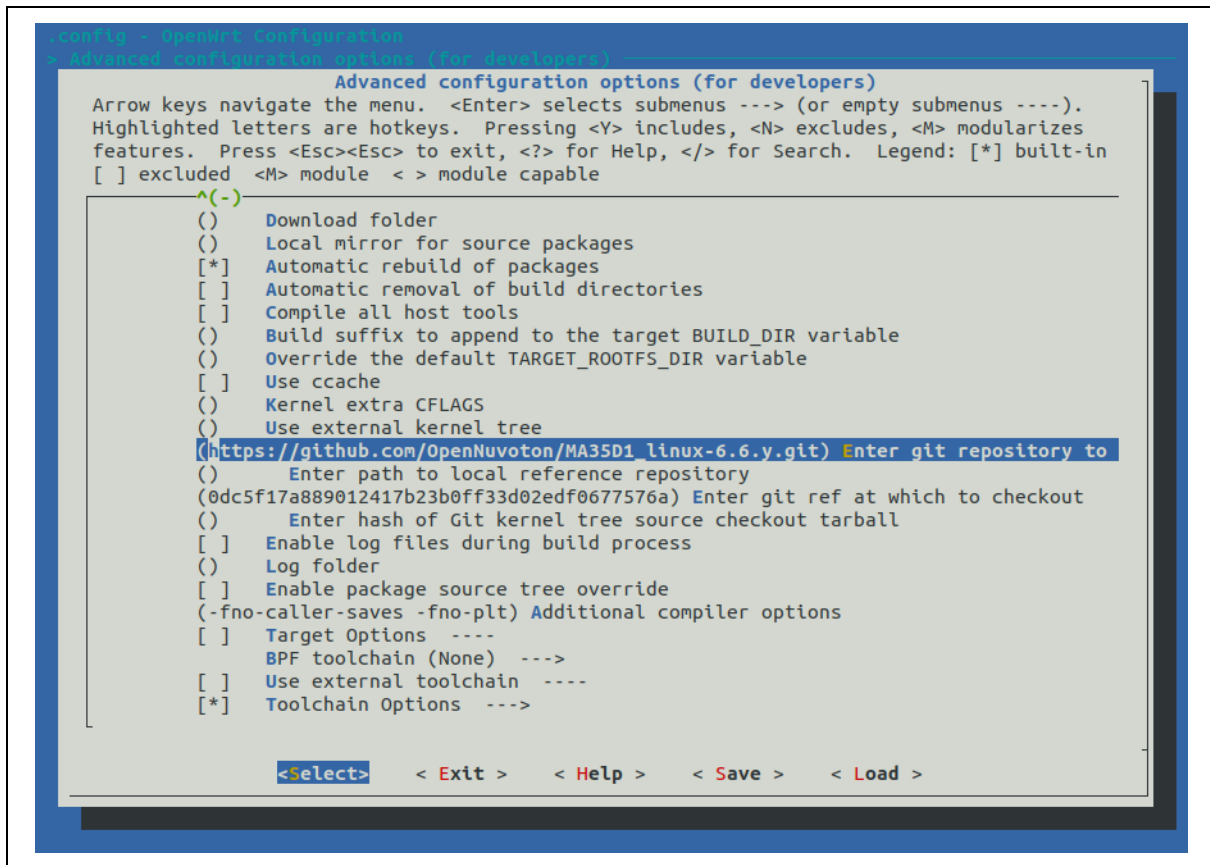


Figure 3-4 Configure Kernel Repository for MA35D1

The default setting uses the HTTPS method to clone git repository from Github, user can change to use other repository managers. Table 3-1 lists available MA35D1 Linux kernel repositories.

Repository Manger	URL
Github	https://github.com/OpenNuvoton/MA35D1_linux-6.6.y.git
Gitlab	https://gitlab.com/OpenNuvoton/Cortex-A-Family/MA35D1_linux-6-6-y.git
Gitee	https://gitee.com/OpenNuvoton/MA35D1_linux-6.6.y.git

Table 3-1 Linux Kernel Repositories for MA35D1

3.3 Linux Kernel Configurations

After setting OpenWrt configurations is completed, execute the command "make kernel_menuconfig" to configure Linux kernel.

```
$ make kernel_menuconfig
```

Before entering to Linux kernel configurations, it will build related toolchain packages and download MA35D1 Linux BSP. This step may take 90 minutes, then user can exit and save the config directly.

If user modifies any Linux kernel configuration, for the MA35D1 IoT board case, the config file will be stored as file Nuvoton-OpenWrt-24.10/target/linux/ma35d1/iot/config-6.6.

3.4 Build Image

3.4.1 MA35D1/MA35D0/MA35D05K/MA35H0 Platform

After configuration is completed, run “make” command to build the OpenWrt. The building may take around 30 minutes.

```
$ make
```

For the MA35D1 IoT board, the generated images are located in the bin/targets/ma35d1/iot/ directory. The output files are shown as Table 3-2.

Image Name	Description
openwrt-ma35d1- $\{\text{Subtarget}\}$ - $\{\text{Profile}\}$ -pack.bin	OpenWrt pack image using NAND / SPINAND / SDCARD / eMMC booting, can be used by MA35D1 nuwriter tool.
openwrt-ma35d1- $\{\text{Subtarget}\}$ - $\{\text{Profile}\}$ -squashfs-firmware.bin	OpenWrt firmware image using NAND / SPINAND / SDCARD / eMMC booting, includes Linux kernel and root filesystem.
openwrt-ma35d1- $\{\text{Subtarget}\}$ - $\{\text{Profile}\}$ -squashfs-sysupgrade.bin	OpenWrt OverlayFS system upgrade image using NAND / SPINAND / SDCARD / eMMC booting with OverlayFS, includes Linux kernel, root filesystem and device tree.
openwrt-ma35d1- $\{\text{Subtarget}\}$ - $\{\text{Profile}\}$ -ext4-sysupgrade.gz	OpenWrt system upgrade image using SDCARD / eMMC booting with ext4-only filesystem, includes Linux kernel, root filesystem and device tree.

Table 3-2 MA35D1 OpenWrt Generated Images in Output Directory

When user changes the Target System, Subtarget or Target Profile setting in configuration, it needs to run below commands to clean the TF-A, Optee-OS and U-boot” packages for re-building, or the loaders will still keep the original setting.

```
$ make package/boot/arm-trusted-firmware-ma35d1/clean
$ make package/boot/optee-os-ma35d1/clean
$ make package/boot/uboot-ma35d1/clean
```

3.4.2 Error Fix

If any error occurs during compilation, use the following command to collect error log for further check.

```
$ make -j1 V=sc
```

3.5 Deploy Image

We provide pack image that includes TF-A, Optee-OS, U-boot, Linux kernel, and file system for NAND / SPINAND / SDCARD / eMMC booting storages. You can use the NuWriter GUI or command to write a pack image into MA35D1 demo board after computer connected that board.

3.5.1 NuWriter GUI Tool

For the NuWriter GUI tool, you need to choose the correct DDR image, and then press “Attach” button to connect the MA35D1 demo board.

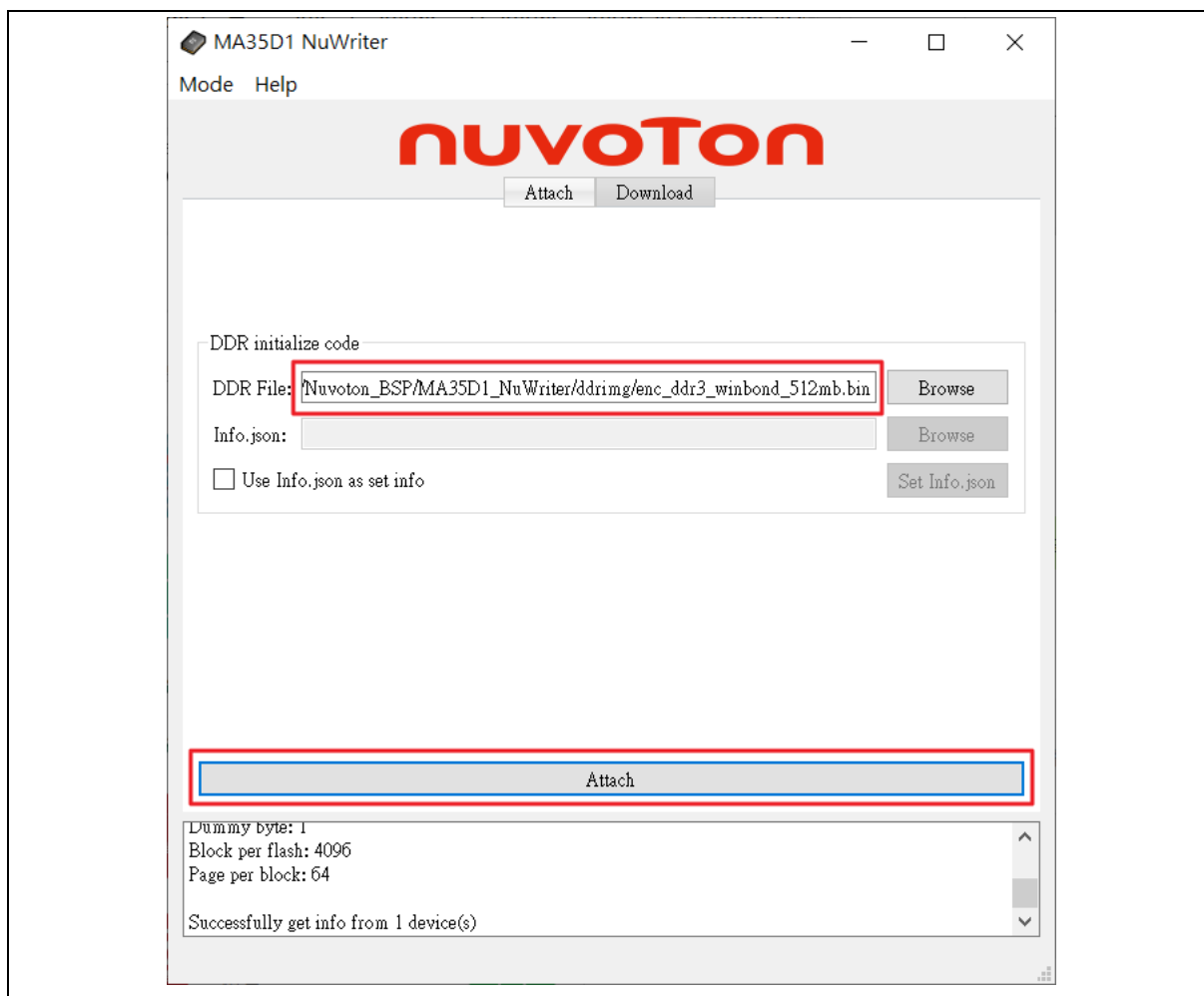


Figure 3-5 Use NuWriter to Attach MA35D1 IoT Board

Select the Download tab and the booting storage. Before to program NAND or SPINAND, you need to erase the storage at first.

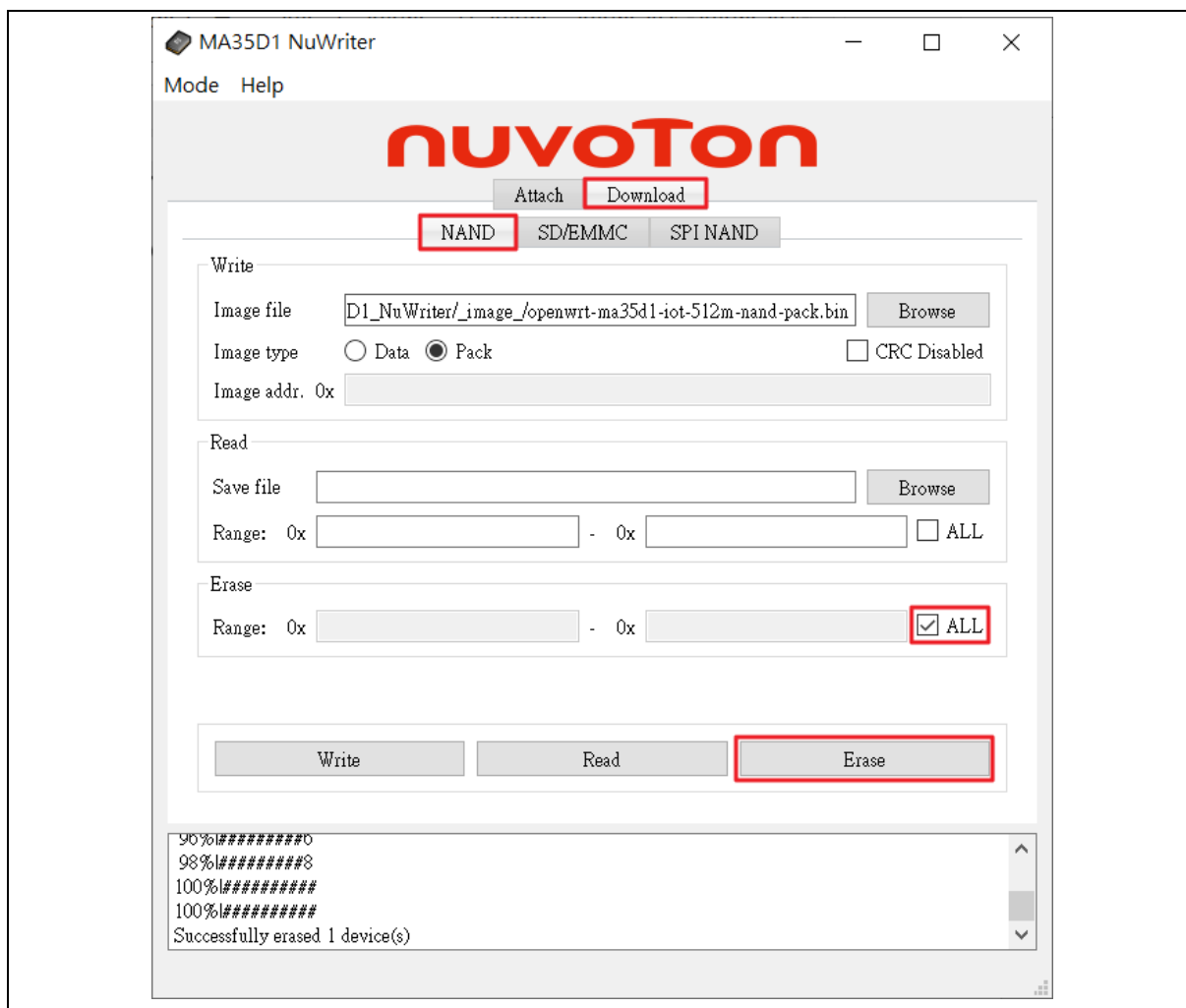


Figure 3-6 Use NuWriter to Erase MA35D1 IoT Board

Choice the correct pack image and select the image type as “Pack”, then click the “Write” button to program the pack image.

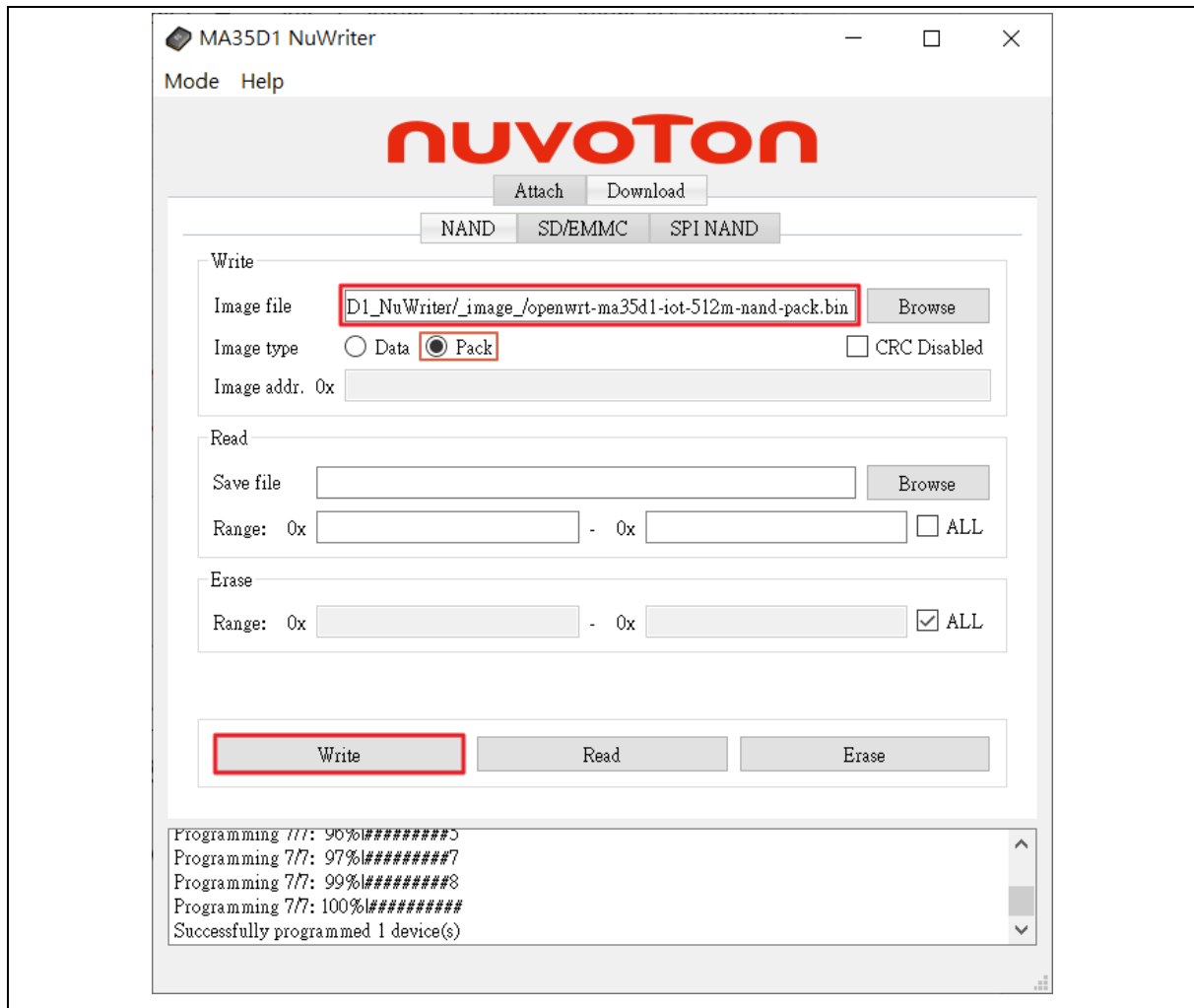


Figure 3-7 Use NuWriter to Program MA35D1 IoT Board

3.5.2 NuWriter Command

For the NuWriter command, you can use “lsusb” command to check whether the connection board is working. **Using “nuwriter” command does not work properly in Docker environment.**

```
$ sudo lsusb
Bus 001 Device 002: ID 80ee:0021 VirtualBox USB Tablet
Bus 001 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
/OpenWrt/bin/targets/ma35d1/iot$
```

Burn openwrt-ma35d1-iot-512m-nand-pack.bin image to NAND.

```
/OpenWrt/bin/targets/ma35d1/iot$ nuwriter.py -a enc_ddr3_winbond_512mb.bin
/OpenWrt/bin/targets/ma35d1/iot$ nuwriter.py -e nand all
/OpenWrt/bin/targets/ma35d1/iot$ nuwriter.py -w nand openwrt-ma35d1-iot-512m-nand-pack.bin
```

Burn openwrt-ma35d1-iot-512m-spinand-pack.bin image to SPINAND.

```
/OpenWrt/bin/targets/ma35d1/iot$ nuwriter.py -a enc_ddr3_winbond_512mb.bin
/OpenWrt/bin/targets/ma35d1/iot$ nuwriter.py -e spinand all
```

```
/OpenWrt/bin/targets/ma35d1/iot$ nuwriter.py -w spinand openwrt-ma35d1-iot-512m-
spinand-pack.bin
```

Burn openwrt-ma35d1-iot-512m-sdcard1-pack.bin image to SDCARD.

```
/OpenWrt/bin/targets/ma35d1/iot$ nuwriter.py -a enc_ddr3_winbond_512mb.bin
/OpenWrt/bin/targets/ma35d1/iot$ nuwriter.py -w sd openwrt-ma35d1-iot-512m-
sdcard1-pack.bin
```

3.6 Secure Boot

This section introduces how to enable secure boot feature in MA35D1/MA35D0/MA35H0 platform.

3.6.1 Configuration

In the “Target Images” menu, user can enable the Secure Boot feature, and change the AES key and ECDSA key.

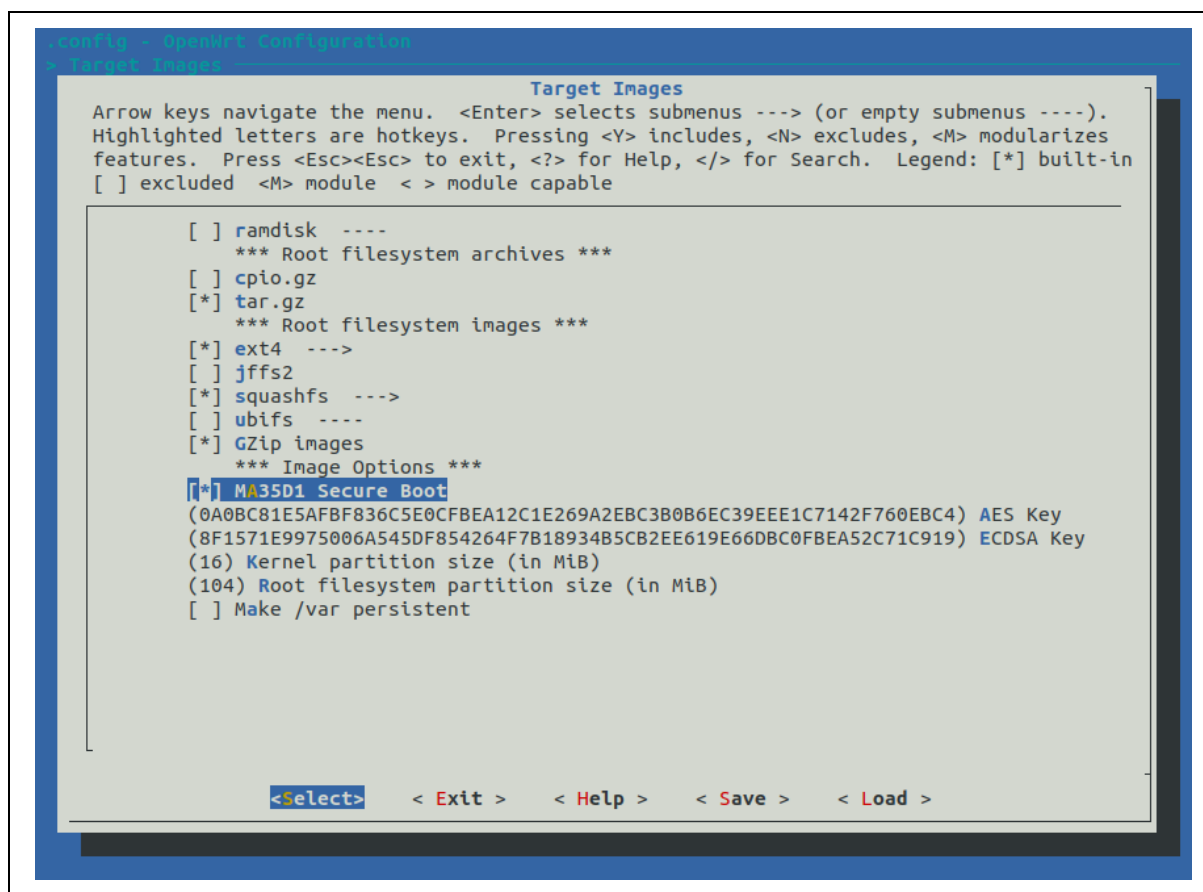


Figure 3-8 Enable the Secure Boot

3.6.2 Build Image

When user changes the Secure Boot setting in configuration, it needs to run below commands to clean the TF-A, Optee-OS and U-boot” packages for re-building, or the loaders will still keep the original setting.

```
$ make package/boot/arm-trusted-firmware-ma35d1/clean
$ make package/boot/optee-os-ma35d1/clean
$ make package/boot/uboot-ma35d1/clean
```

After the image building is done, user can find the output images with the “-enc” ending, which is different to the general output images. Except the original images, the OTP key file will also be generated. Following is the case of IoT board with NAND booting.

```
openwrt-ma35d1-iot-512m-nand-pack-enc.bin
openwrt-ma35d1-iot-512m-nand-squashfs-sysupgrade-enc.bin
openwrt-ma35d1-otp-key-enc.json
```

The content of the OTP key file is as following.

```
$ cat openwrt-ma35d1-otp-key-enc.json
{
  "publicx": "72F84F681092E3A05C1437E3E40534962A5C70556025D348FF9DB97D6AF83EB5",
  "publicy": "8D32DAC7AB6F90332E8E0060E159E0B31502BB4FB2D78369F02D1F5B0C335AD3",
  "aeskey": "0A0BC81E5AFBF836C5E0CFBEA12C1E269A2EBC3B0B6EC39EEE1C7142F760EBC4"
}
```

3.6.3 Deploy OTP Key File

For secure boot mode, except the pack image, the OTP key file also needs to be programmed to the key store of MA35D1 through NuWriter tools. For more details, please refer to *MA35D1 Secure Boot Application Note*.

3.6.3.1 NuWriter GUI Tool

For the NuWriter GUI tool, you need to change to OTP mode, and choice the OTP key file, then click the “Write” button to program the pack image.

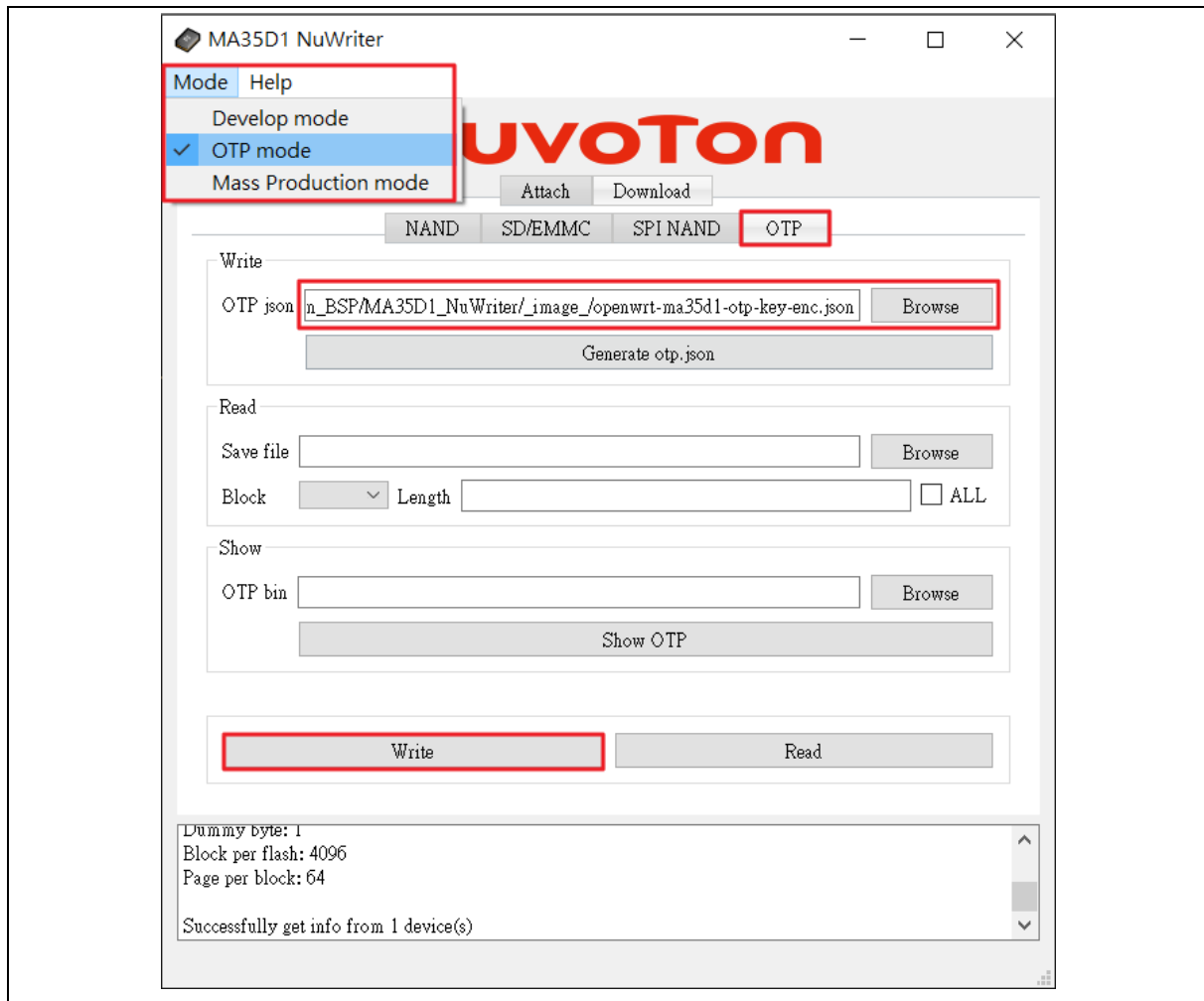


Figure 3-9 Use NuWriter to Program MA35D1 OTP key

3.6.3.2 NuWriter Command

For the NuWriter command, you can add follow command to burn OTP key file to Key Store.

```
/OpenWrt/bin/targets/ma35d1/iot$ nuwriter.py -w otp openwrt-ma35d1-otp-key-enc.json
```

3.7 emWin (TODO)

This section introduces how to enable the emWin feature on supported Nuvoton boards. Currently, the following development boards equipped with a display panel can support emWin.

NuMaker-HMI-MA35D1

NuMaker-HMI-MA35H0

3.7.1 Copy the emWin Source to OpenWrt

After downloading the emWin package from the Nuvoton website, the user can extract the Linux emWin package tarball. For example, in the MA35D1 case, the user can execute the following commands.

```
$ cd package/nuvoton/emwin-ma35d1/
$ tar zxvf MA35D1_Linux_emWin_Package_20240110.tar.gz
```

The directory names after decompression contain a date suffix, and the user needs to remove it.

```
mv MA35D1_Linux_emWin_Package_20240110 MA35D1_Linux_emWin_Package
```

Please be aware that OpenWrt only supports the emWin package version released after the year 2024.

3.7.2 Configuration

In the “Nuvoton” menu, user can enable the emWin and tslib features.

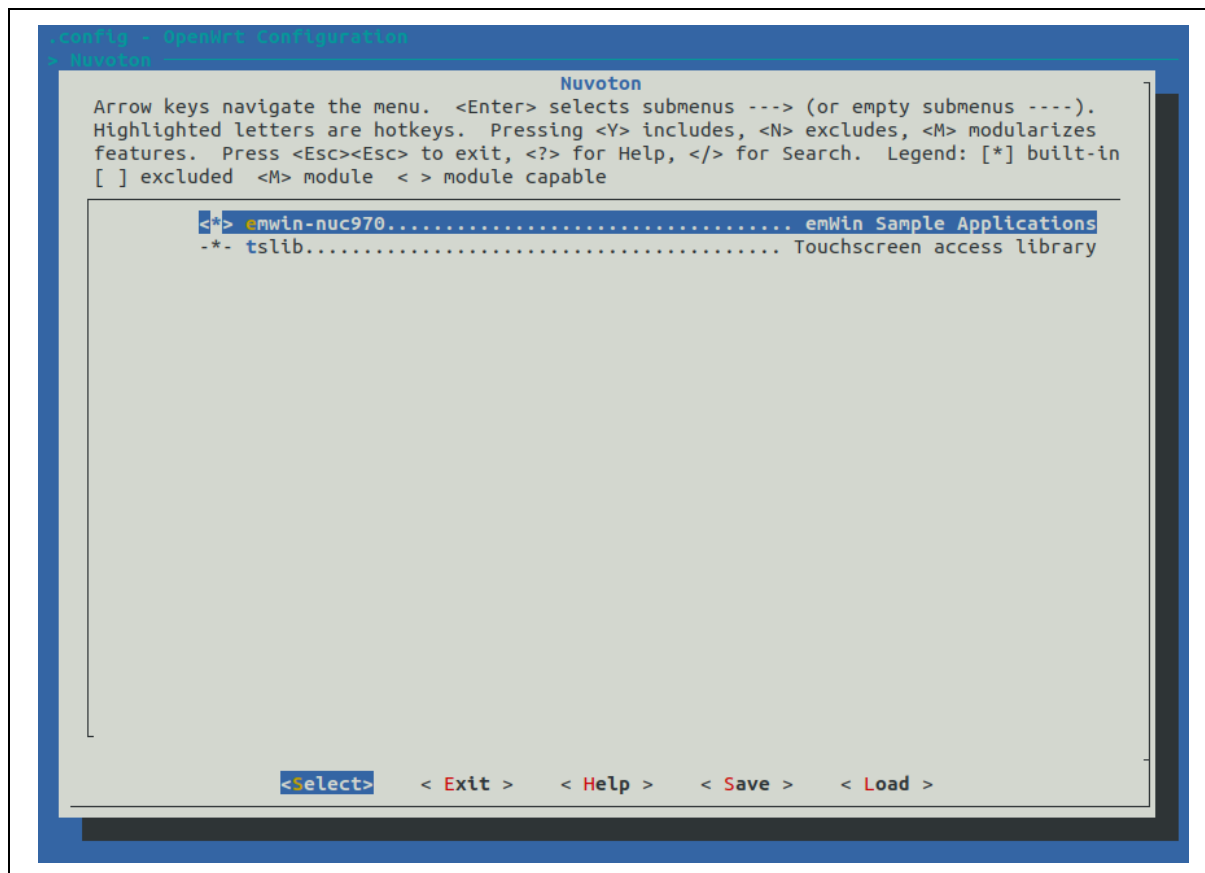


Figure 3-10 Enable the Secure Boot

3.7.3 Samples

The emWin source package contains several samples under “Sample” folder.

```
$ ls package/nuvoton/emwin-ma35d1/MA35D1_Linux_emWin_Package/Sample/
GUIDemo SimpleDemo SimpleDemoAppWizard
```

In the image-building procedure, these emWin samples will be compiled and copied to the rootfs. After OpenWrt boots up, users can run the emWin demos as follows.

```
root@OpenWrt:/# ls usr/bin/*Demo*
usr/bin/GUIDemo
usr/bin/SimpleDemoAppWizard
usr/bin/SimpleDemo
```

4 TEST OPENWRT

4.1 Booting Messages

After starting the device, user will see the OpenWrt booting messages, as shown in Figure 4-1.

```
[ 20.035498] NET: Registered PF_PPPOX protocol family
[ 20.047200] kmodloader: done loading kernel modules from /etc/modules.d/*
[ 20.372467] urngd: v1.0.2 started.
[ 21.840149] ma35d1-audio sound: snd_soc_register_card() failed: -517
[ 21.846651] platform sound: deferred probe pending
[ 26.531150] nuvoton-dwmac 40120000.ethernet eth0: Register MEM_TYPE_PAGE_POOL RxQ-0
[ 26.604909] nuvoton-dwmac 40120000.ethernet eth0: PHY [stmmac-0:00] driver [RTL8211F Gigabit Ethernet] (irq=POLLI)
[ 26.624663] nuvoton-dwmac 40120000.ethernet eth0: No Safety Features support found
[ 26.632234] nuvoton-dwmac 40120000.ethernet eth0: No MAC Management Counters available
[ 26.640140] nuvoton-dwmac 40120000.ethernet eth0: IEEE 1588-2008 Advanced Timestamp supported
[ 26.648631] nuvoton-dwmac 40120000.ethernet eth0: configuring for phy/rgmii-id link mode
[ 28.753898] nuvoton-dwmac 40120000.ethernet eth0: Link is Up - 100Mbps/Full - flow control off

BusyBox v1.36.1 (2025-06-23 20:40:36 UTC) built-in shell (ash)

 _   _          _ 
| |_| |        | |
| |_| |__      | |__
| |_| |__      | |__
|_| |_|       |_|_|
  W I R E L E S S   F R E E D O M

-----
OpenWrt 24.10.2, r28739-d9340319c6
===== WARNING! =====
There is no root password defined on this device!
Use the "passwd" command to set up a new password
in order to prevent unauthorized SSH logins.
-----
root@OpenWrt:~#
```

Figure 4-1 OpenWrt Booting Messages

4.2 Network Settings

To get the network setting information, user can run the following command.

```
uci show network
```

The default network setting is using the DHCP address, as shown in Figure 4-2 Default Network Settings

```
root@OpenWrt:/#  
root@OpenWrt:/# uci show network  
network.loopback=interface  
network.loopback.ifname='lo'  
network.loopback.proto='static'  
network.loopback.netmask='255.0.0.0'  
network.lan=interface  
network.lan.ifname='eth0'  
network.lan.proto='dhcp'  
network.lan.netmask='255.255.255.0'  
root@OpenWrt:/#
```

Figure 4-2 Default Network Settings

To change the network settings, user can modify the file path `/etc/config/network` directly, or run the “uci set” command. For example, to change to a static address, user can run following commands to modify and reset the network settings.

```
uci set network.lan.proto=static
uci set network.lan.ipaddr=192.168.1.100
uci set network.lan.netmask=255.255.255.0
/etc/init.d/network restart
```

4.3 LuCI Web Interface

To login the LuCI Web interface, user can connect to https://YOUR_IP_ADDRESS through a web browser such as Chrome. In the first-time login, user may encounter a security warning message, as shown in Figure 4-3 First Time to Login the LuCI Web Interface

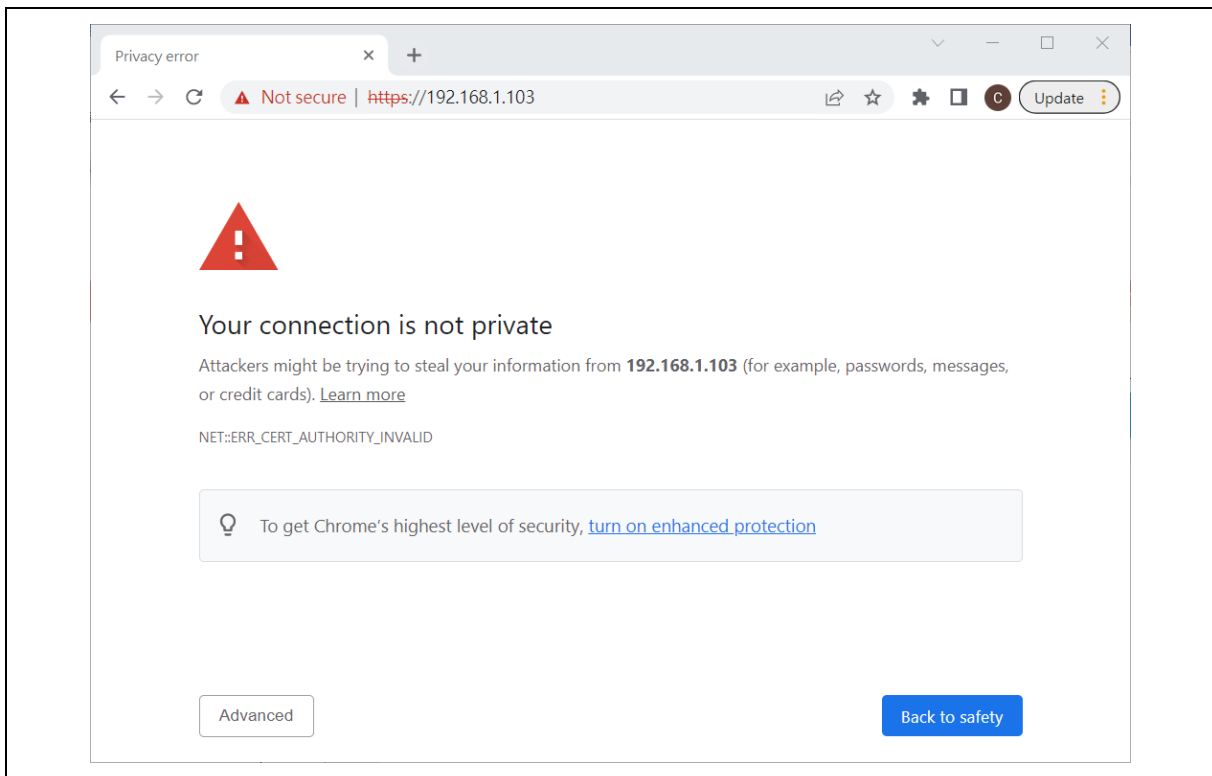


Figure 4-3 First Time to Login the LuCI Web Interface

After clicking the “Advanced” button, user will see a new screen, as shown in Figure 4-4 Proceed with Connection to LuCI

. Please proceed with the connection.

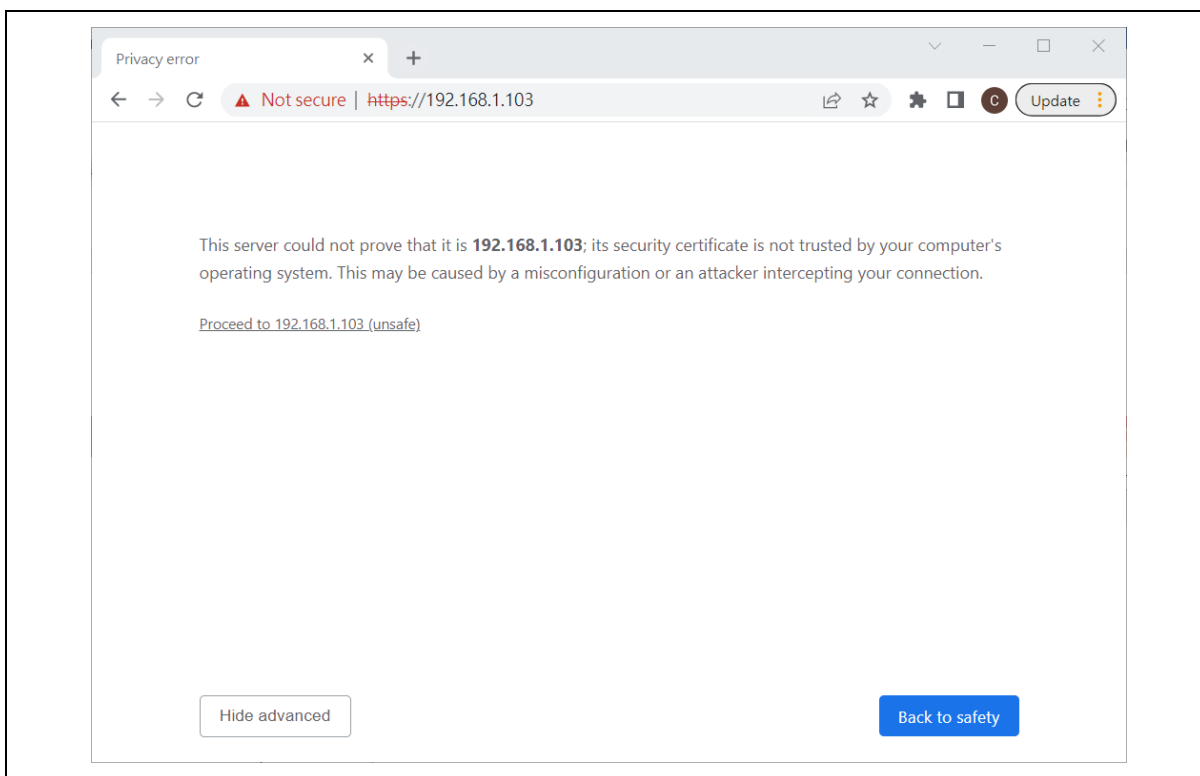


Figure 4-4 Proceed with Connection to LuCI

Then user can see the login screen. Since there is no password by default, user can login directly, or set a new password, as shown in Figure 4-5 Login Screen of LuCI

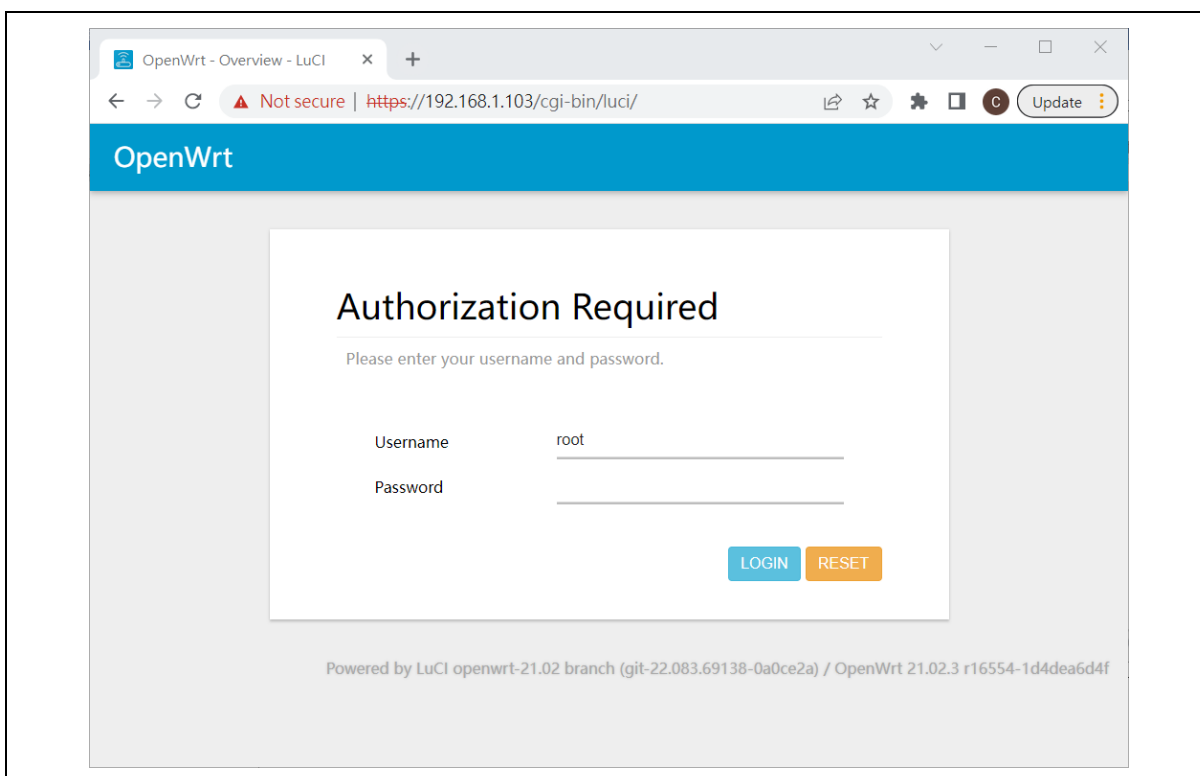


Figure 4-5 Login Screen of LuCI

4.4 Firmware Update

With the LuCI interface, user can upgrade the image which includes Linux kernel, root filesystem and device tree.

For the MA35D1 IoT board, the images are as follows.

```
openwrt-ma35d1-iot-512m-nand-squashfs-sysupgrade.bin
openwrt-ma35d1-iot-512m-spinand-squashfs-sysupgrade.bin
openwrt-ma35d1-iot-512m-sdcard1-squashfs-sysupgrade.gz
openwrt-ma35d1-iot-512m-sdcard1-ext4-sysupgrade.gz
```

To do the OpenWrt firmware upgrade, enter "System -> Backup/Flash Firmware". Choose the new sysupgrade image file, and then click "FLASH IMAGE" button, as shown in Figure 4-6 Firmware Upgrade in LuCI

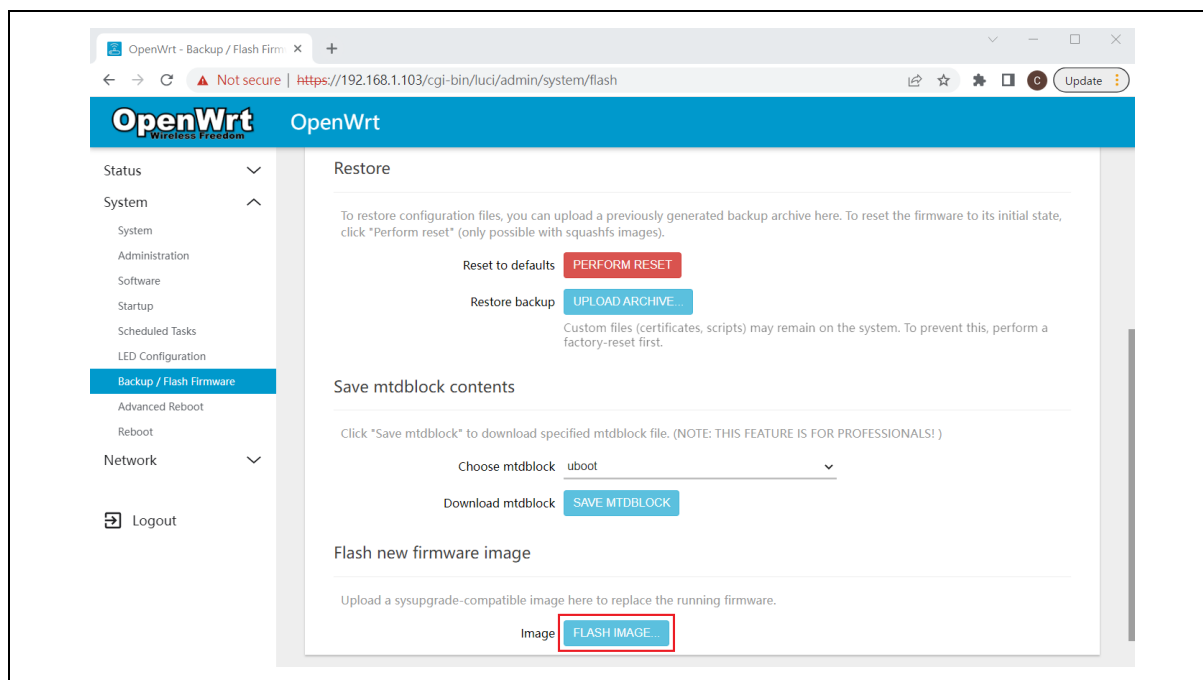


Figure 4-6 Firmware Upgrade in LuCI

5 REVISION HISTORY

Date	Revision	Description
2026.01.09	1.00	- Initial version. - Support MA35D1 platform.
2026.06.09	1.01	Support MA35D0 and MA35D05K platforms.
2026.08.07	1.02	- Firmware update supports Linux device-tree. - SDCARD boot supports squashfs+ext4 overlay.

Important Notice

Nuvoton Products are neither intended nor warranted for usage in systems or equipment, any malfunction or failure of which may cause loss of human life, bodily injury or severe property damage. Such applications are deemed, "Insecure Usage".

Insecure usage includes, but is not limited to: equipment for surgical implementation, atomic energy control instruments, airplane or spaceship instruments, the control or operation of dynamic, brake or safety systems designed for vehicular use, traffic signal instruments, all types of safety devices, and other applications intended to support or sustain life.

All Insecure Usage shall be made at customer's risk, and in the event that third parties lay claims to Nuvoton as a result of customer's Insecure Usage, customer shall indemnify the damages and liabilities thus incurred by Nuvoton.

*Please note that all data and specifications are subject to change without notice.
All the trademarks of products and companies mentioned in this datasheet belong to their respective owners.*